

Basic (venv, start to finish) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way (macOS / Linux)

```
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
cat requirements.txt
```

Expected output (from the final `cat requirements.txt`):

```
certifi==2024.8.30
charset-normalizer==3.3.2
idna==3.8
requests==2.32.3
urllib3==2.2.3
```

The version numbers may differ on your machine — `pip freeze` lists `requests` itself plus the smaller packages it depends on internally (`certifi`, `charset-normalizer`, `idna`, `urllib3`), which is why there are 5 lines instead of 1.

This version is correct and complete for the exercise as stated.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — with the Windows variant

macOS / Linux:

```
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
```

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install requests
pip freeze > requirements.txt
```

Expected output, either platform, after `pip freeze > requirements.txt` : no console output (it's redirected into the file). Open `requirements.txt` to see the same 5 lines as the Basic Version above.

Approach 2 — verifying each step as you go

```
python -m venv .venv
ls .venv
# bin/  include/  lib/  pyvenv.cfg

source .venv/bin/activate
which python
# /path/to/your/project/.venv/bin/python

pip install requests
pip show requests
# Name: requests
# Version: 2.32.3
# Summary: Python HTTP for Humans.
# ...
```

```
pip freeze > requirements.txt
wc -l requirements.txt
# 5 requirements.txt
```

Difference from Basic: both Intermediate approaches run the exact same 4 commands as Basic — nothing about the actual environment changes. Approach 1 just adds the Windows equivalent, since `source` and `.venv/bin/activate` are Linux/macOS-specific paths. Approach 2 adds a check after each command (`ls`, `which python`, `pip show`, `wc -l`) so you catch a problem — like activation silently failing — immediately, right where it happened, instead of discovering it 3 steps later when `pip freeze` produces an empty or wrong file.

The most common failure: running `source .venv/bin/activate` from the wrong folder (so `.venv/bin/activate` doesn't exist relative to where you are), or forgetting the `source` keyword on macOS/Linux (`.venv/bin/activate` alone won't work in most shells — it needs to be *sourced* into your current shell, not just executed). Check with `pwd` that you're in the project folder first, and check `(.venv)` actually appears in your prompt after activating.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a `Makefile` that automates the 4 commands

```
.PHONY: setup
setup:
    python -m venv .venv
    .venv/bin/pip install requests
    .venv/bin/pip freeze > requirements.txt
    @echo "Setup complete. Activate with: source .venv/bin/activate"
```

Run it with:

```
make setup
```

Expected output:

```
python -m venv .venv
.venv/bin/pip install requests
Collecting requests
  Downloading requests-2.32.3-py3-none-any.whl (64 kB)
...
Successfully installed certifi-2024.8.30 charset-normalizer-3.3.2 idna-3.8 reques
.venv/bin/pip freeze > requirements.txt
Setup complete. Activate with: source .venv/bin/activate
```

Notice it calls `.venv/bin/pip` directly rather than activating first — each line of a `Makefile` recipe runs in its own separate shell, so an `activate` on one line wouldn't stay active for the next line anyway. Calling the venv's own `pip` binary by its full path avoids that problem entirely.

Approach 2 — a shell script with error handling

```
#!/usr/bin/env bash
set -e

python -m venv .venv
source .venv/bin/activate
pip install requests
```

```

pip freeze > requirements.txt

echo "Done. requirements.txt has $(wc -l < requirements.txt) package(s)."
```

Run it with:

```

chmod +x setup.sh
./setup.sh
```

Expected output:

```
Done. requirements.txt has 5 package(s).
```

`set -e` at the top means the script stops immediately the moment any line in it fails (for example, if `pip install requests` fails because of no internet connection), instead of continuing on and silently producing a broken `requirements.txt`.

Approach 3 — `uv`, a modern all-in-one tool

```

uv init myproject
cd myproject
uv add requests
```

Expected output (abridged — exact timings and hashes will differ):

```

Initialized project `myproject`
Resolved 5 packages in 12ms
Prepared 5 packages in 234ms
Installed 5 packages in 8ms
+ certifi==2024.8.30
+ charset-normalizer==3.3.2
+ idna==3.8
+ requests==2.32.3
+ urllib3==2.2.3
```

`uv` creates the `.venv` folder for you automatically, and instead of a `requirements.txt` you maintain by hand with `pip freeze`, it writes a `uv.lock` file every time you `uv add` something — a lockfile that pins the exact versions of everything, dependencies included, so `uv sync` on any other machine recreates the identical environment. There's no separate "activate" step needed either — `uv run python script.py` runs inside the right environment automatically.

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate still requires a human to type 4 commands correctly, in order, every time. All 3 Advanced approaches remove that requirement, but in different ways. Approach 1 (`Makefile`) is the most minimal — it just replays your existing 4 commands under one name, with no new concepts. Approach 2 (shell script) is similar but adds real error handling (`set -e`), which a `Makefile` recipe also has in a different form (`make` stops on the first failing line by default too). Approach 3 (`uv`) is the most different of the three — it's not automating your 4 commands, it's replacing the whole `venv + pip + pip freeze` workflow with a single faster tool that manages the lockfile for you, so you never run `pip freeze` by hand at all.

Which one should you actually write? For this exercise, and for any solo script or small project, the Basic Version's 4 commands typed by hand are completely fine — you're not automating anything you don't already understand. The moment more than one person (or a CI pipeline) needs to set up the same project, reach for Advanced Approach 1 (a `Makefile`) — it's the smallest change that removes "did I type that right" as a source of bugs. Advanced Approach 3 (`uv`) is worth learning next, on your own time outside this exercise, since exact lockfile-based dependency management is what most new Python projects are moving toward — but understanding the plain `venv + pip` workflow first (as this exercise has you do) is what makes `uv` make sense once you get there, instead of feeling like a black box.